4.24 Cache——地址映射（直接映射习题课）

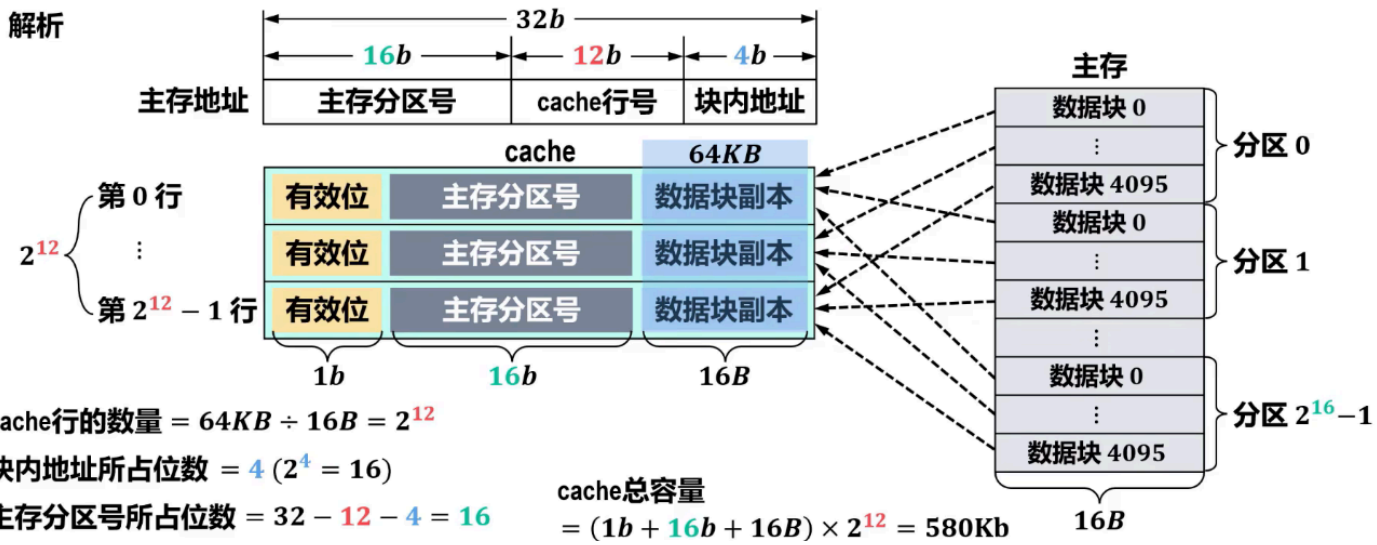
考纲定位：计算机组成原理 · 第 4 章 存储器（4.24） | 重要度 ★★★★★ | 常见考法：大题（地址划分、Cache 容量、Tag/Index/Offset、命中率、数组访问）

一句话核心：先拆地址，再算行号与 Tag，最后算容量；数组题先看访问顺序是否利用空间局部性

图旁说明：题图与解析图保留；文字只给“考什么 / 常见错因 / 解题模板”，不转写图中解答

例题一

【例题1】假设计算机cache系统采用直接映射方式，主存地址为32位，按字节编址，cache数据区大小为$64KB$，主存数据块大小为$16B$。请给出主存地址构成细节，并计算cache总容量。



考什么：直接映射的地址划分与 Cache 总容量 **常见错因**

- 把 Cache 容量直接当行数，忘记除以块大小
- Tag 位数用总地址位数减 Offset，漏减 Index
- 总容量只算数据，忘记 Tag/有效位/脏位

解题模板

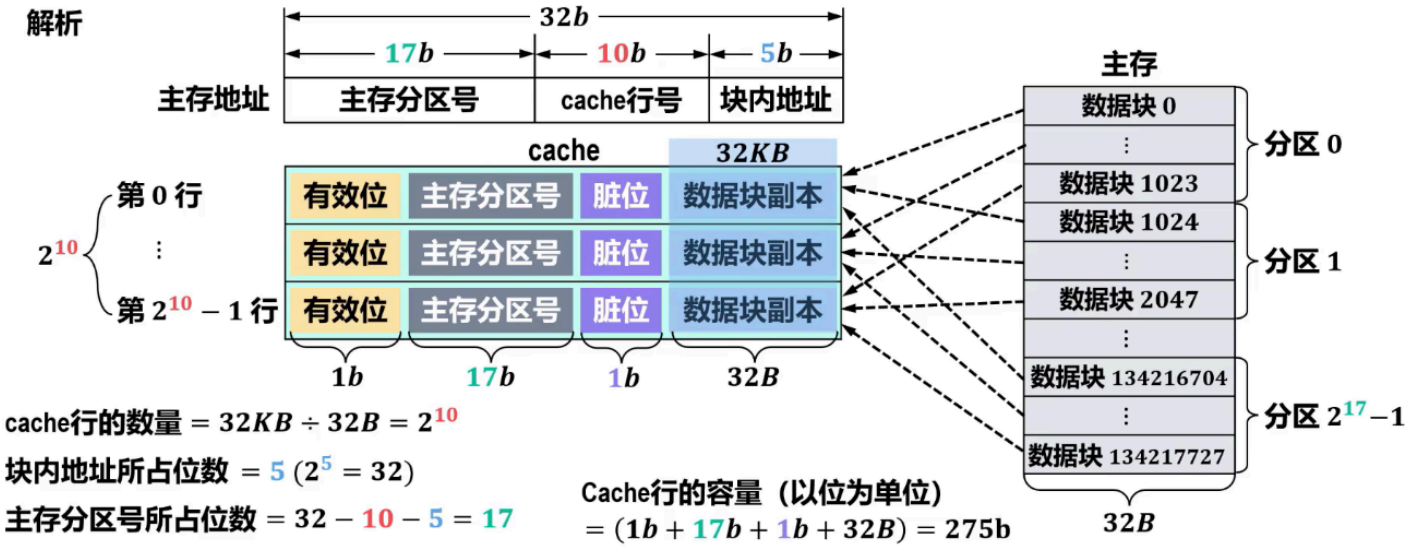
1. 行数 = Cache 数据容量 / 块大小
2. Offset = $\log_2$(块大小)
3. Index = $\log_2$(行数)
4. Tag = 地址位数 − Index − Offset
5. 总容量 = 行数 × (数据位 + Tag + 有效位 + 脏位)

例题二

【2021年 题16】若计算机主存地址为32位，按字节编址，Cache数据区大小为$32KB$，主存块大小为$32B$，采用直接映射方式和回写（Write Back）策略，则Cache行的位数至少是（A）。

A. 275　　B. 274　　C. 258　　D. 257

解析



cache行的数量 $= 32KB \div 32B = 2^{10}$

块内地址所占位数 $= 5\ (2^5 = 32)$

主存分区号所占位数 $= 32 - 10 - 5 = 17$

Cache行的容量（以位为单位）$= (1b + 17b + 1b + 32B) = 275b$

考什么：写回策略的脏位对 Cache 总容量的影响 **常见错因**

- 忘记写回策略需要 1 位脏位
- 把脏位算进数据位
- 有效位/脏位按字节算，而不是按行算

解题模板

1. 先按直接映射算 Tag/Index/Offset
2. 写回策略：每行加 1 位脏位
3. 每行开销 = Tag + 有效位 1 + 脏位 1
4. 总容量 = 行数 × (数据位 + 开销位)

例题三

第一问

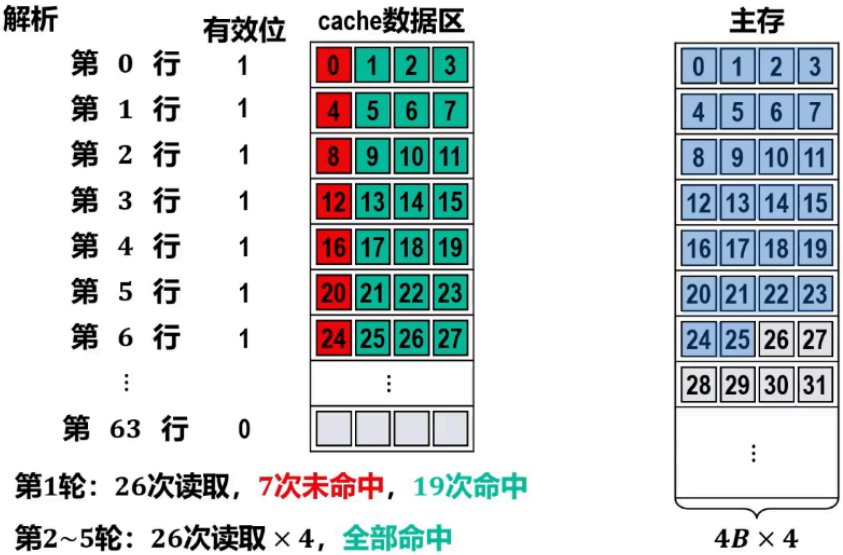
地址映射 —— 直接映射（习题课）

【例题2】

假设计算机字长为32位，主存按字节编址，cache采用直接映射方式，cache数据区大小为$1KB$，主存数据块大小为4个字。试回答以下问题：

1. 设cache的初始状态为空，若CPU要依次从主存0到25号单元读出26个字（假设访问主存1次读出1个字），并重复此顺序5次，计算cache访问的命中率。
2. 若主存访问时间为$100ns$，cache访问时间为$10ns$，根据（1）中计算出的命中率计算cache/主存系统的平均访问时间T_a。
3. 计算cache/主存系统的访问效率e。

解析



第1轮：26次读取，7次未命中，19次命中

第2~5轮：26次读取$\times 4$，全部命中

命中率$h = \frac{26\times5-7}{26\times5} \approx 94.6\%$

第二问

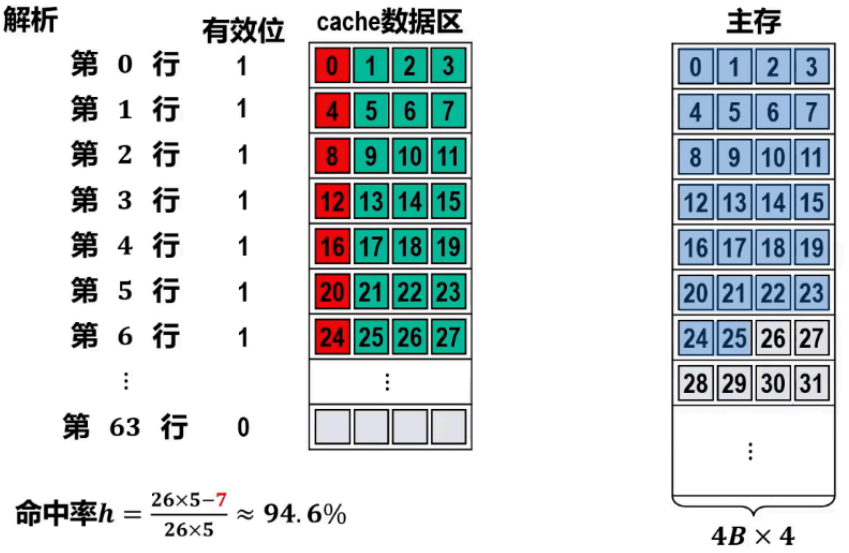
地址映射 —— 直接映射（习题课）

【例题2】

假设计算机字长为32位，主存按字节编址，cache采用直接映射方式，cache数据区大小为$1KB$，主存数据块大小为4个字。试回答以下问题：

1. 设cache的初始状态为空，若CPU要依次从主存0到25号单元读出26个字（假设访问主存1次读出1个字），并重复此顺序5次，计算cache访问的命中率。
2. 若主存访问时间为$100ns$，cache访问时间为$10ns$，根据（1）中计算出的命中率计算cache/主存系统的平均访问时间T_a。
3. 计算cache/主存系统的访问效率e。

解析



命中率$h = \frac{26\times5-7}{26\times5} \approx 94.6\%$

平均访问时间$T_a = 10ns \times 94.6\% + 100ns \times (1 - 94.6\%)$
$= 14.86ns$

第三问

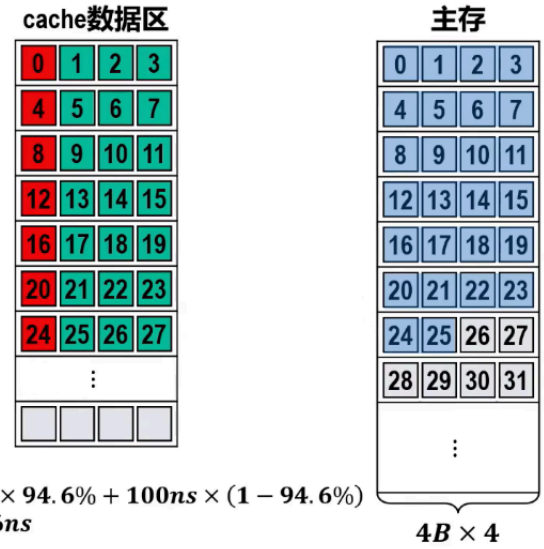
地址映射 —— 直接映射（习题课）

【例题2】

假设计算机字长为32位，主存按字节编址，cache采用直接映射方式，cache数据区大小为$1KB$，主存数据块大小为4个字。试回答以下问题：

1. **设cache的初始状态为空，若CPU要依次从主存0到25号单元读出26个字（假设访问主存1次读出1个字），并重复此顺序5次，计算cache访问的命中率。**
2. **若主存访问时间为$100ns$，cache访问时间为$10ns$，根据（1）中计算出的命中率计算cache/主存系统的平均访问时间T_a。**
3. **计算cache/主存系统的访问效率e。**

解析



平均访问时间$T_a = 10ns \times 94.6\% + 100ns \times (1 - 94.6\%) = 14.86ns$

访问效率$e = \dfrac{10ns}{14.86ns} \approx 67.3\%$

考什么：直接映射的行号计算、Tag 比较、命中判断 **常见错因**

- 用主存地址直接比较，而不是用 Tag
- 忘记有效位
- 行号算错，导致比较了错误的行

解题模板

1. 由地址取 Index → 定位行
2. 取 Tag → 与该行 Tag 比较
3. 有效位 = 1 且 Tag 相等 → 命中
4. 否则缺失，按块调入并更新 Tag/有效位

例题四

地址映射 —— 直接映射（习题课）

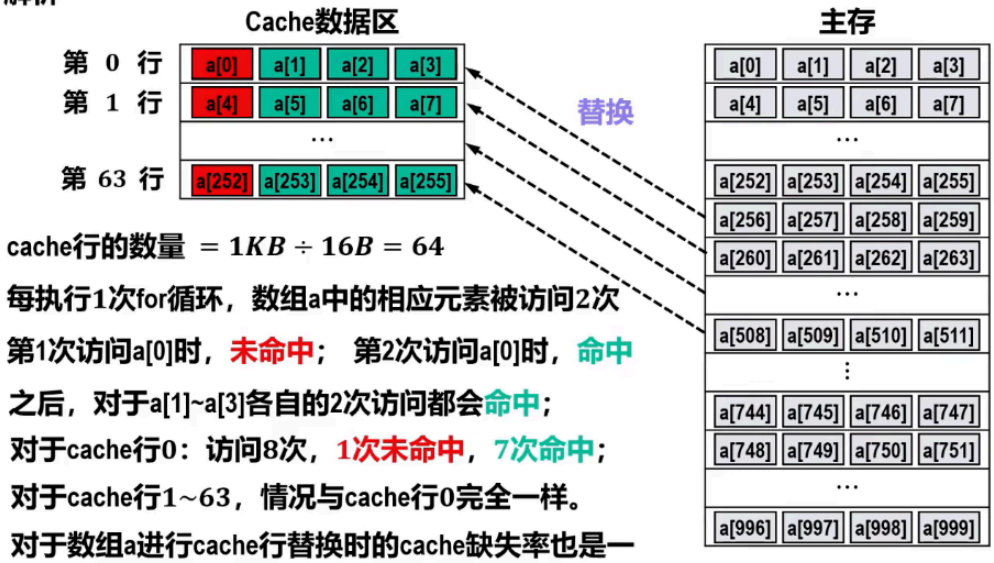
【2016年 题15】
有如下 C 语言程序段：

```
for ( k=0; k<1000; k++ )
    a[k] = a[k] + 32;
```

若数组a和变量k均为int型、int型数据占4B，数据Cache采用直接映射方式，数据区大小为1KB、块大小为16B，该程序执行前Cache为空，则该程序段执行过程中访问数组a的Cache缺失率约为（C）。

A. 1.25%　　B. 2.5%
C. 12.5%　　D. 25%

解析



cache行的数量 $= 1KB \div 16B = 64$

每执行1次for循环，数组a中的相应元素被访问2次

第1次访问a[0]时，未命中； 第2次访问a[0]时，命中

之后，对于a[1]~a[3]各自的2次访问都会命中；

对于cache行0：访问8次，1次未命中，7次命中；

对于cache行1~63，情况与cache行0完全一样。

对于数组a进行cache行替换时的cache缺失率也是一样的，即1次未命中，7次命中，cache缺失率为1/8。

考什么：直接映射的冲突缺失与替换 **常见错因**

- 认为直接映射可以任意选择替换行
- 忽略多个主存块映射到同一行
- 把“冲突”当成“容量不足”

解题模板

1. 算每个主存块的行号
2. 行号相同的块互相冲突
3. 直接映射只能替换该固定行
4. 冲突缺失多，是直接映射的主要缺点

例题五

【2010年 题44】某计算机的主存地址空间大小为$256MB$，按字节编址。指令Cache和数据Cache分离，均有8个Cache行，每个Cache行大小为$64B$，数据Cache采用直接映射方式。
现有两个功能相同的程序A和B，其伪代码如下：

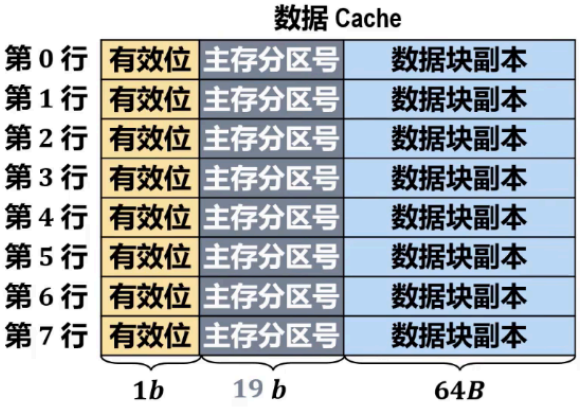
程序A：
```
int a[256][256]
……
int sum_array1 ( )
{
    int i, j, sum=0;
    for ( i = 0; i < 256; i++)
       for ( j = 0; j < 256; j++)
            sum += a[i][j];
    return sum;
}
```

程序B：
```
int a[256][256]
……
int sum_array2 ( )
{
    int i, j, sum=0;
    for ( j = 0; j < 256; j++)
       for ( i = 0; i < 256; i++)
            sum += a[i][j];
    return sum;
}
```

假定int类型数据用32位补码表示，程序编译时i、j、sum均分配在寄存器中，数组a按行优先方式存放，其首地址为320（十进制数）。回答下列问题，要求说明理由或给出计算过程。

1. 若不考虑用于Cache一致性维护和替换算法的控制位，则数据Cache的总容量为多少？
2. 数组元素a[0][31]和a[1][1]各自所在的主存块对应的Cache行号分别是多少（Cache行号从0开始）？
3. 程序A和B的数据访问命中率各是多少？哪个程序的执行时间更短？

第一问



每个Cache行的容量 $= (1 + 19 + 64 \times 8)/8$ 字节

数据Cache的总容量 $= (1 + 19 + 64 \times 8)/8$ 字节 $\times 8$

$= 532$ 字节



由于主存按字节编址，地址空间大小为$256MB = 2^{28}B$

因此，主存地址位宽 $= 28$

由于数据Cache有8行 $= 2^3$行

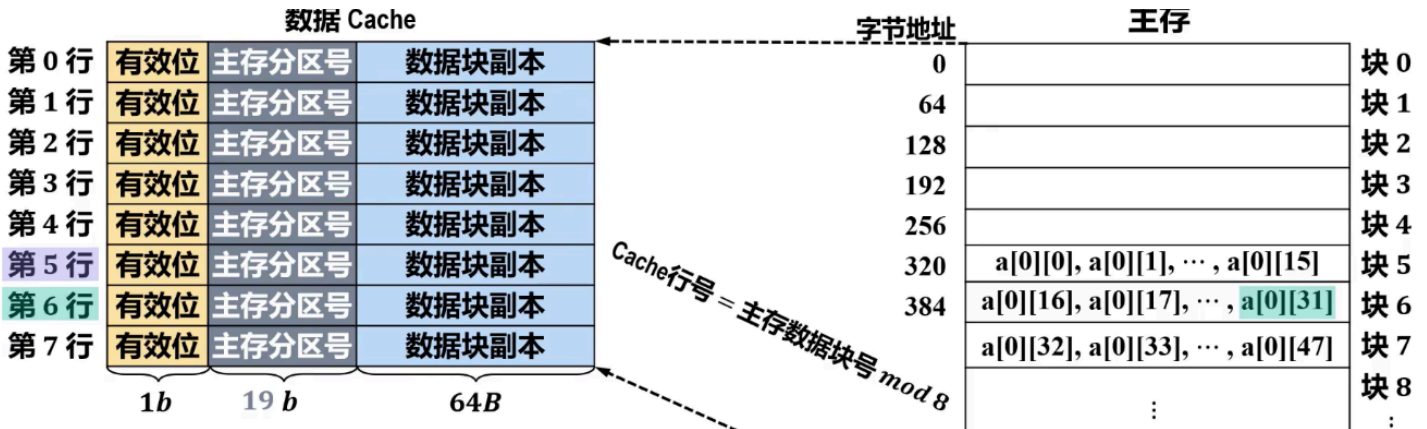
因此，Cache行号所占位数 $= 3$

由于每个Cache行中的数据块副本大小为$64B = 2^6B$

因此，块内地址所占位数 $= 6$

主存分区号所占位数 $= 28 - 3 - 6 = 19$

第二问



a[0][31]的主存地址 $= 320 + 31 \times 4 = 444$

a[0][31]所在主存数据块的块号 $= \lfloor 444 \div 64 \rfloor = 6$

a[0][31]所在主存数据块对应的Cache行号 $= 6 \bmod 8 = 6$

a[1][1]的主存地址 $= 320 + 256 \times 4 + 1 \times 4 = 1348$

a[1][1]所在主存数据块的块号 $= \lfloor 1348 \div 64 \rfloor = 21$

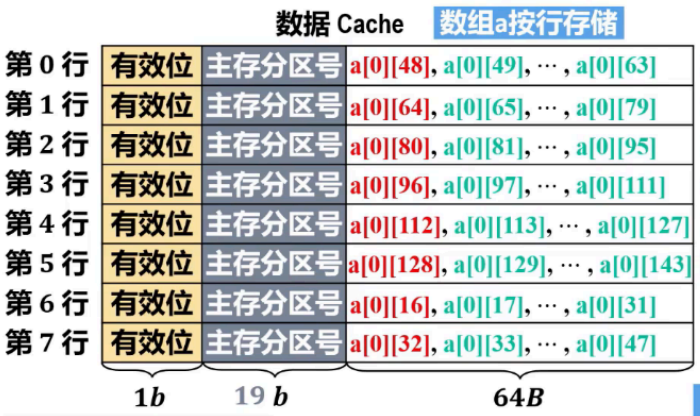
a[1][1]所在主存数据块对应的Cache行号 $= 21 \bmod 8 = 5$

或站在主存地址角度

a[0][31]的主存地址 $= 320 + 31 \times 4 = 444 = (1\,1011\,1100)_2$ Cache行号 $= (110)_2 = 6$

a[1][1]的主存地址 $= 320 + 256 \times 4 + 1 \times 4 = 1348 = (101\,0100\,0100)_2$ Cache行号 $= (101)_2 = 5$

第三问：程序A 按行访问



第1次读取缺失，载入主存内相应数据块，后15次读取都会命中

第1次读取缺失，载入主存内相应数据块，后15次读取都会命中

第1次读取缺失，载入主存内相应数据块，后15次读取都会命中

第1次读取缺失，载入主存内相应数据块，后15次读取都会命中

第1次读取缺失，载入主存内相应数据块，后15次读取都会命中

第1次读取缺失，替换为主存内相应数据块，后15次读取都会命中

第1次读取缺失，载入主存内相应数据块，后15次读取都会命中

第1次读取缺失，载入主存内相应数据块，后15次读取都会命中

程序A：（按行访问）

```
int a[256][256]
……
int sum_array1 ( )
{
    int i, j, sum=0;
    for ( i = 0; i < 256; i++)
        for ( j = 0; j < 256; j++)
            sum += a[i][j];
    return sum;
}
```

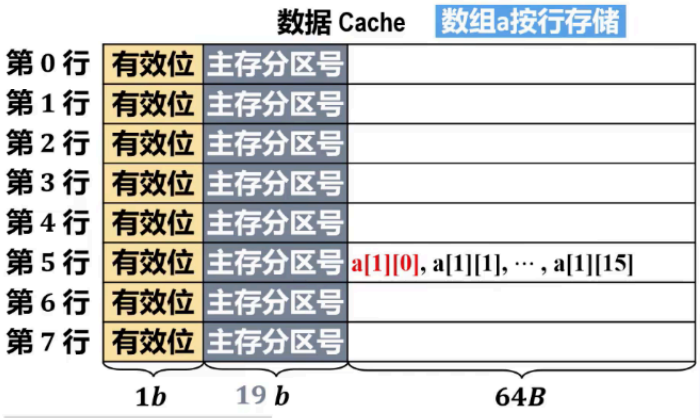
程序编译时i、j、sum均分配在寄存器中，因此程序A和B的数据访问命中率只考虑数组a即可。

不难发现，程序A中对数组a的访问都符合上述规律，因此命中率h计算如下：

$$h = \frac{15}{16} = 93.75\%$$

程序A中数组a的访问顺序与存储顺序相同，具有较好的空间局部性。

程序B 按列访问



第2次读取缺失，替换为主存内相应数据块

在执行内层for循环时，不同行的同一列数组元素对应同一个cache行，例如： a[0][0], a[1][0], ⋯ , a[255][0]每个都对应cache行 5。第1次读取a[0][0]缺失，载入主存中相应数据块到cache行5，第2次读取a[1][0]缺失，主存中相应数据块替换到cache行 5 ，依此类推，数组a中的所有元素都无法命中，因此命中率$h = 0$。

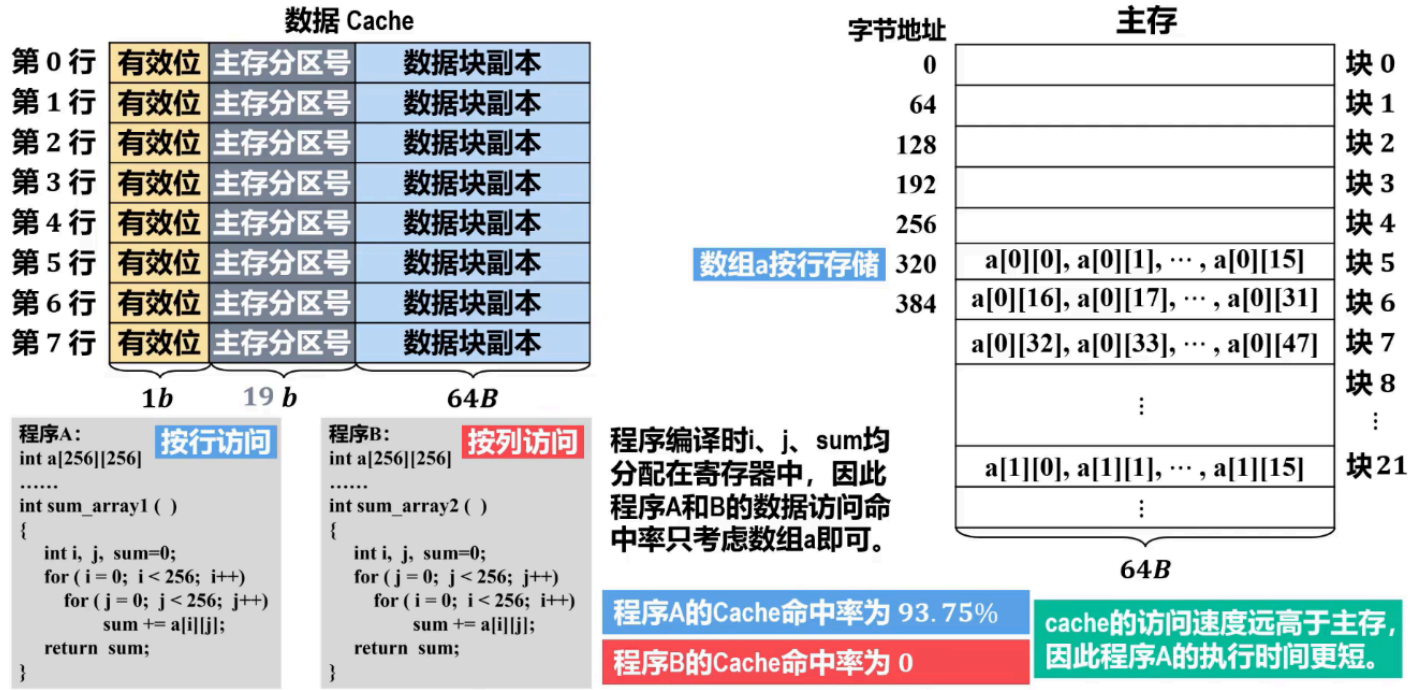
程序B： 按列访问

```
int a[256][256]
……
int sum_array2 ( )
{
    int i, j, sum=0;
    for ( j = 0; j < 256; j++)
        for ( i = 0; i < 256; i++)
            sum += a[i][j];
    return sum;
}
```

程序编译时i、j、sum均分配在寄存器中，因此程序A和B的数据访问命中率只考虑数组a即可。

读取序列： a[0][0], a[1][0], ⋯ , a[15][0]
a[16][0], a[17][0], ⋯ , a[31][0]
⋮

对比



程序A： 按行访问

```
int a[256][256]
……
int sum_array1 ( )
{
    int i, j, sum=0;
    for ( i = 0; i < 256; i++)
        for ( j = 0; j < 256; j++)
            sum += a[i][j];
    return sum;
}
```

程序B： 按列访问

```
int a[256][256]
……
int sum_array2 ( )
{
    int i, j, sum=0;
    for ( j = 0; j < 256; j++)
        for ( i = 0; i < 256; i++)
            sum += a[i][j];
    return sum;
}
```

程序编译时i、j、sum均分配在寄存器中，因此程序A和B的数据访问命中率只考虑数组a即可。

程序A的Cache命中率为 93.75%

程序B的Cache命中率为 0

cache的访问速度远高于主存，因此程序A的执行时间更短。

考什么：数组访问顺序与空间局部性对命中率的影响 **常见错因**

- 只看数组大小，不看访问顺序
- 认为按列访问也一定能命中
- 忽略块大小与行大小的关系

解题模板

1. 确定数组元素大小、块大小、每块能放几个元素
2. 按行访问：连续元素在同一块，空间局部性好
3. 按列访问：跨行跳跃，块利用率低，冲突/缺失多
4. 比较两种访问的命中率与总访问时间

知识关联：直接映射（4.23 Cache——地址映射（直接映射））； Cache 基本概念（4.21 Cache——基本概念）；替换算法（4.26 Cache——替换算法）